

Лабораторна робота №9

Тема:

Документування API, UI та автоматизація документації в CI/CD

Мета роботи

Метою лабораторної роботи є:

- формування у студентів **контрактного мислення** при проєктуванні API;
- закріплення підходу **API-first** та використання специфікації **OpenAPI**;
- набуття практичних навичок роботи з екосистемою **Swagger**;
- ознайомлення з принципами **візуальної документації frontend**;
- інтеграція документації у **CI/CD-процеси** як частини Definition of Done.

Передумови виконання

Студент повинен мати:

- виконані **ЛР-7** (архітектура документації);
- виконані **ЛР-8** (Documentation as Code);
- визначені функціональні вимоги (SSD);
- базові знання REST API та Git.

Завдання лабораторної роботи

Студенту необхідно **реалізувати комплексну систему документації**, яка включає:

- **контрактну API-документацію**;

- інтерактивну Swagger-документацію;
- приклад візуальної UI-документації;
- автоматизовану публікацію документації.

Порядок виконання роботи

1. Документування API як контракту

Необхідно:

1. Визначити роль API у системі:
 - internal / public;
 - для frontend / mobile / third-party.
2. Описати **призначення API** в документації.

Приклад

API надає контракт для frontend-застосунку з керування замовленнями та користувачами. API є внутрішнім і використовується SPA-клієнтом.

2. Формування API за підходом API-first

До реалізації визначити:

- ендпоінти;
- HTTP-методи;
- структури запитів та відповідей;
- коди помилок.

Приклад

POST /api/orders

GET /api/orders/{id}

PATCH /api/orders/{id}

DELETE /api/orders/{id}

3. Створення OpenAPI-специфікації

Створити OpenAPI-специфікацію у форматі **YAML** або **JSON**.

Обов'язкові розділи

- info (назва, опис, версія);
- paths;
- components/schemas;
- responses;
- security (за наявності).

Приклад фрагмента

paths:

/api/orders/{id}:

get:

summary: Get order by ID

parameters:

- name: id

in: path

required: true

schema:

type: string

responses:

'200':

description: Order found

'404':

description: Order not found

4. Swagger та автоматична генерація

Необхідно:

1. Використати **Swagger Editor** для валідації специфікації.
2. Згенерувати **Swagger UI** для перегляду та тестування API.
3. *(Опційно, але рекомендовано):*
 - згенерувати клієнтський SDK (Swagger Codegen);
 - або описати генерацію документації безпосередньо з коду.

Приклад опису

Swagger UI дозволяє виконувати тестові запити до API без використання сторонніх клієнтів.

5. Візуальна документація frontend (Storybook)

Необхідно:

1. Обрати **1 – 2 UI-компоненти** (реальні або демонстраційні).
2. Описати їх у форматі **Stories**:
 - базовий стан;
 - альтернативний стан (error / disabled / loading).

Приклад

Button:

- Default
- Disabled
- Loading

6. Проектування автоматизації документації в CI/CD

Необхідно **спроектувати та формально описати** процес автоматизації документації в межах CI/CD для свого проекту.

Студент повинен:

1. Визначити **артефакти документації**, які підлягають автоматичній генерації:
 - API-документація (Swagger UI)
 - UI-документація (Storybook)
 - загальний сайт документації (з ЛР-8)
2. Визначити **тригери генерації**:
 - push у feature-гілку;
 - merge у main (або develop).
3. Описати **послідовність кроків pipeline**:
 - збірка документації;
 - валідація (lint / schema validation);
 - публікація.
4. Вказати **місце публікації документації**:
 - GitHub Pages або GitLab Pages;
 - структура URL.
5. Описати **політику versioning документації**:
 - відповідність версії документації версії продукту;
 - правила створення нової версії.

Форма виконання

Опис автоматизації подається у вигляді:

- Markdown-документа docs/ci-cd-docs.md
- **або** діаграми pipeline + текстового пояснення

Приклад (фрагмент)

Documentation CI/CD Pipeline

Trigger:

- Merge into `main`

Steps:

1. Validate OpenAPI specification
2. Build Swagger UI
3. Build Storybook
4. Generate static documentation site
5. Publish to GitHub Pages

Versioning:

- Documentation version matches application version
- Breaking API changes require a new major documentation version

7. Формалізація Definition of Done для документації

Необхідно **розробити та формалізувати Definition of Done (DoD)** для змін у проєкті з точки зору документації.

Студент повинен:

1. Визначити **типи змін**, для яких DoD застосовується:

- зміна API;
- додавання нового ендпоінта;
- зміна UI-компонента;
- нефункціональні зміни.

2. Для кожного типу змін описати:

- які **документаційні артефакти** повинні бути оновлені;
- які **автоматизовані перевірки** мають пройти.

3. Сформулювати **перевірювані критерії готовності**:

- що саме має існувати;
- у якому вигляді;
- де це перевіряється.

Форма виконання

Definition of Done оформлюється у вигляді:

- Markdown-документа docs/definition-of-done.md
- або таблиці з чіткими правилами

Приклад (табличний формат)

Тип зміни	Обов'язкові дії	Перевірка
Додано API-ендпоінт	Оновити OpenAPI	OpenAPI validation
Змінено response	Оновити Swagger UI	Manual / CI
Додано UI-компонент	Додати Story	Storybook build
Breaking change	Нова версія документації	Version tag

Приклад (текстовий формат)

Definition of Done (Documentation)

- Any API change must be reflected in the OpenAPI specification
- Swagger UI must be regenerated successfully
- UI changes require updated or new Storybook stories
- Documentation build must pass without errors

Вимоги до оформлення та задачі

- Формат: **Git-репозиторій**
- OpenAPI-файл
- Swagger UI (локально або demo)
- Опис Storybook
- Markdown-документ з описом CI/CD і DoD

Критерії оцінювання

Критерій	Частка
Контрактна коректність API	25%
Якість OpenAPI-специфікації	20%
Swagger та автоматизація	20%
UI-документація (Storybook)	20%
CI/CD та Definition of Done	15%

Контрольні питання

1. Чому API-документація є контрактом?
2. У чому перевага API-first?
3. Яку роль відіграє Swagger UI?
4. Навіщо потрібен Storybook?
5. Чому документація має бути частиною Definition of Done?